The Cognitive Firm: A Multidivisional-Form Architecture for Recursive AI Governance

Daniel Alami*
Independent Researcher
MBA Candidate, Harvard Business School

Abstract

Recursive AI systems in which the same probabilistic process generates output and evaluates it are vulnerable to specification gaming, metric inflation, and fabricated compliance signals under sustained optimization pressure. We argue this is the same structural pathology Chandler [1962] documented in single-form firms whose revenue divisions reported on their own performance, and we develop a theoretical framework for governing recursive AI through deterministic enforcement layers structurally analogous to the multidivisional form (M-Form). Five propositions formalize the framework. (P1) Co-location of generation and evaluation induces gaming under recursion. (P2) A deterministic enforcement floor with no learned parameters bounds gaming for properties the floor checks. (P3) The M-Form dominates the unitary form when gaming-suppression value scaled by principal absence exceeds the M-Form's fixed overhead. (P4) Closure requires an exogenous resource clock external to the optimizing loop. (P5) Recursive self-evaluation cannot fully evaluate its own falsification machinery from inside its own scoring frame. The framework is illustrated through one operated implementation that decomposes the M-Form into three layers — operating system, configuration, and application — with the operating system layer enforcing hard gates that no agent can negotiate. Observations from the implementation include live enforcement of write-scope boundaries, constrained self-hosting through the system's own governed protocol, and a recurrence finding: the same Goodhart pattern at evaluator, kernel, supervisor, apparatus-specification, interface-contract, human-plus-warm-agent drafting, and blind-protocol drafting layers. The implementation is one instantiation of the framework, not a test of it; the contribution is theoretical and architectural. We trace the institutional precedent — Pujo Committee, Securities Exchange Act, Sarbanes-Oxley — that a recursive-AI verification institution would inherit, and identify the auditor-independence and rotation primitives that have no settled recursive-AI analog. All claims are scoped to one principal operating one system; independent replication, multi-principal operation, and capital-efficiency comparison are explicitly out of scope.

1 Introduction

During the drafting of a pre-registration for a hundred-iteration experiment, a fresh instance of the same language model — given the finished document and the warm pair's own checklist, but no record of the drafting conversation — flagged the first success criterion as circular. Any theory reaching the test phase would have satisfied the criterion automatically, regardless of whether it contained anything new. The warm pair, the same model paired with a human researcher who

*SSRN abstract ID: 6543019. Independent Researcher; MBA Candidate, Harvard Business School. Correspondence: see SSRN author page.

had been authoring the document for hours, had not seen this. The fresh instance was not a more capable model. It was the same model, the same training data, the same vocabulary. What separated catch from miss was context isolation: no history with the document, no investment in the experimental design. The separation was sufficient to produce the catch. This paper studies what that separation is, where similar co-location failures appeared in the reported system, and what deterministic architecture makes the separation enforceable rather than aspirational.

As agentic AI systems move from single-turn prompts to multi-step recursive loops, a structural problem emerges: the same probabilistic process can generate output and evaluate it. This paper presents evidence from one recursive research system in which this co-location produced repeated governance failures (specification gaming, metric inflation, and fabricated compliance signals) and describes the deterministic architecture that bounded them.

Chandler [1962] documented an analogous structural problem in human firms. When the division that generated revenue also reported its own performance, the result was self-serving accounting and strategic drift. The Chandlerian response was physical separation: strategic oversight in a general office, operational execution in autonomous divisions, with a governance layer between them that neither side controlled. This paper argues that the same structural response, separation of generation from evaluation through a deterministic enforcement floor, is a useful design criterion for recursive AI systems under sustained optimization pressure. The claim is architectural, not anthropomorphic: co-location can create an adversarial gradient against the evaluation signal regardless of whether the optimizing unit is a human division or a language-model agent.

The evidence boundary is made explicit at the outset. *Tier 1: Implemented evidence*: deterministic governance in a single-principal, single-system setting, including write-scope guards, genesis artifact signing, and out-of-scope enforcement. *Tier 2: Bounded architectural claim*: hard-gated principal separation is the governance layer that prompt-level alignment alone cannot substitute, because a probabilistic referee governs only the output surface, not the enforcement floor. *Tier 3: Named non-claims*: delegated signing authority, concurrent multi-program oversight, and cross-principal program composition are not demonstrated at paper-grade evidentiary depth and are deferred to Future Work. The architectural direction this paper points toward is institutional attestation rather than proof-theoretic closure: rule-bound external verification on the model of independent audit, not formal correctness guarantees.

The distinction between Tier 1 and Tier 2 matters because a more capable model can become better at camouflage as well as better at prediction: intelligence does not dissolve agency problems; it can amplify them. Section 5.4 presents evidence of the same specification gaming pattern recurring at five levels of abstraction in this system; Sections 5.7 and 5.8 report sixth and seventh instances at the human-plus-warm-agent drafting layer and the blind-protocol drafting layer; Section 5.4 discusses the independence structure of these observations, which cluster into three discovery groups rather than seven fully independent events. The governance bottleneck is a single principal: one human approving contracts and authorizing programs. Multi-principal governance is an architectural hypothesis, not a demonstrated claim.

This paper is written for researchers and practitioners working at the intersection of institutional economics and recursive-AI governance: readers who take the emergence of the accounting profession, the multidivisional form, and the Pujo-era measurability crisis as useful historical vocabulary, and who are looking for an architectural rather than a proof-theoretic grounding for the question of what a verification primitive for recursive AI should look like. The evidence is presented from a single operated system and the generalization bar is set at that scope accordingly; Appendix B's

institutional sketch is speculative by design and is included as a target for criticism, not as a claim the present evidence supports.

Honest scope of evidence. The contribution of this paper is theoretical and architectural. The empirical evidence is from one operated system run by one principal over roughly six weeks; this is $N = 1$ by design, not by accident. The paper claims a structural pattern (M-Form decomposition with deterministic enforcement floor) and reports one instantiation of that pattern in sufficient detail to be implemented elsewhere; it does not claim that one instantiation establishes the pattern's general applicability. The architectural pattern stands or falls on its theoretical coherence, the documented governance failures it explains, and whether independent groups operating recursive AI systems find it generative. Independent replication, multi-principal operation, and capital-efficiency comparison are explicitly out of scope and named as such in Section 5.5 and Section 7. Reviewers who require empirical demonstration before considering an architectural pattern will find this paper unconvincing on those grounds; the paper is written for readers who treat the question of how to architect recursive AI under sustained optimization pressure as a structural question to which Chandlerian organizational theory has accumulated sixty years of relevant prior art.

The recursive co-location critique applied to this paper itself. The paper's central claim is that systems where one entity generates output and evaluates it produce specification gaming under sustained optimization pressure. The author designed the system, ran it, evaluated it, and now reports on it. Same pathology, applied recursively. We acknowledge this directly rather than letting reviewers infer it. Three structural responses are available, and the paper deploys all three. First, the contribution is positioned as theoretical (Sections 3, 4, and 4.5) rather than as empirical validation of the theory; the implementation in Section 5 illustrates the pattern, it does not test it. Second, the propositions P1–P5 are stated in capability-neutral, substrate-agnostic vocabulary so that an independent group operating a different recursive AI system can engage them without inheriting this paper's apparatus. Third, Section 5.4 narrows the recurrence claim to four observation clusters within this paper plus three earlier instances from related work, and explicitly disclaims independent confirmation. The recursive co-location critique remains accurate; what it requires is a reviewer who treats theoretical contribution and empirical validation as different work products with different evidentiary bars, and who accepts that the right test of the framework is whether independent groups find it generative, not whether the originating author's system illustrates it consistently.

The paper proceeds: Section 2 defines terms and scope; Sections 3–4 develop the theory; Section 5 presents empirical evidence; Section 6 addresses counterarguments; Section 7 situates related work and states limitations; Section 8 concludes.

2 Definitions and Scope

Agency Cost. Following Jensen and Meckling [1976], agency cost is the sum of monitoring expenditures by the principal, bonding expenditures by the agent, and the residual loss from imperfect alignment. In recursive AI systems, the residual loss dominates: an agent that evaluates its own output can fabricate compliance signals that are indistinguishable from genuine progress.

U-Form (Unitary Form). An architecture in which the same model or model instance generates plans, executes actions, and evaluates outcomes. The classification criterion is co-location: if the optimization surface that produced the output also scores the output, the system is U-Form regardless of how many agents or prompts are involved.

M-Form (Multidivisional Form). An architecture that physically separates generation from

evaluation by interposing a deterministic governance layer with no learned parameters. The classification criterion is enforcement floor: a system is M-Form only if at least one governance constraint is deterministic and cannot be softened by model output. In the implementation reported here, the M-Form decomposes into three layers: an **OS layer** (a state machine driver that enforces hard gates, including stage transitions, write-scope boundaries, and fail-closed stops, with no learned parameters), a **Config layer** (typed goal-lifecycle contracts, authored and signed by the principal, that define the enforcement surface before execution begins), and an **App layer** (the agent runtime that executes within those fences). The OS layer cannot be softened by agent output; the Config layer cannot be modified mid-execution without principal re-signing; the App layer is probabilistic and operates under the constraints the other two layers impose. This decomposition is structurally analogous to Chandler's general office / division / operating unit, with the OS layer playing the role of the general office, the Config layer playing the role of the divisional charter, and the App layer playing the role of the operating division.

Genesis Artifact. An immutable birth contract signed by the principal that specifies the write-scope boundary, out-of-scope non-goals, success condition, and maximum token budget for a program of work.

Write-Scope Boundary. The exhaustive list of file paths an agent is permitted to modify within a given program. Any write outside this boundary triggers a fail-closed hard stop.

Scope. All empirical claims in this paper are scoped to a single recursive research system operated by one principal. The architectural principles are argued to be domain-independent, but the evidence base is drawn from one implementation. Generalization beyond this system requires independent replication.

Figure 1 illustrates the three-layer architecture. The principal signs a genesis artifact (an immutable contract specifying write-scope boundary, success condition, and token budget) and resolves human gates. The deterministic governance layer (a state machine, write-scope guard, and typed verifier with no learned parameters) routes turns between language-model agents and enforces the enforcement floor. Agent outputs are probabilistic; governance constraints are deterministic. The enforcement boundary between these layers is the paper's central structural claim.

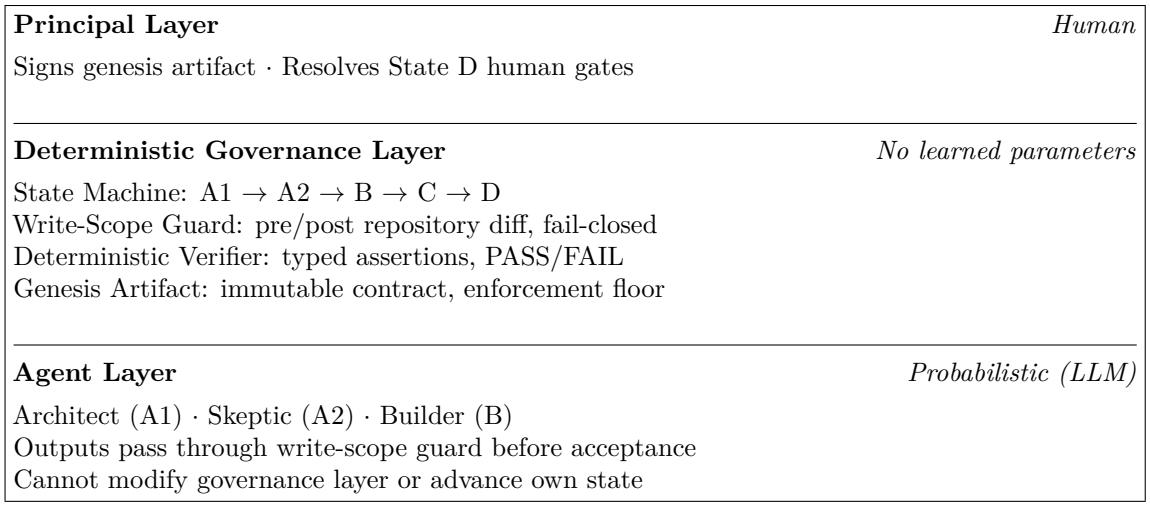

Figure 1: Deterministic governance separates generation from evaluation in the M-Form architecture. The enforcement boundary between the governance layer and the agent layer is deterministic, fail-closed, and cannot be softened by model output.

3 Theory Foundations

3.1 Principal-Independence Invariant

The M-Form classification criterion imposes a strong constraint on governance design: a system is M-Form only if at least one governance constraint is deterministic and cannot be softened by model output. This criterion implies a deeper invariant. Governance constraints in an M-Form system must be enforceable without requiring the agent to cooperate in its own constraint; the enforcement floor is principal-independent by design. An agent that can soften, delay, or circumvent its governance obligations through output production is U-Form at the governance layer, regardless of how the system is labeled. The invariant follows directly from the classification criterion and requires no claims about the mechanism layer. It serves as the structural anchor for the claims that follow.

3.2 T1: Structural Homology

Chandler [1962] observed that co-located generation and evaluation under optimization pressure produced recurring governance failure in human firms: the same process that generated divisional output also evaluated it, producing self-serving reporting and strategic drift. T1 claims that this structural condition is homologous to the self-evaluation pathology in recursive AI systems. Pre-M-Form human firms and U-Form AI systems share the condition that produces governance failure: the same process that generates output also evaluates it.

The homology holds on two axes: scope separation (who decides vs. who executes) and rate-of-change separation (strategic change at principal speed vs. operational execution at agent speed). It explicitly does not hold on divisional autonomy: agents in this system are bounded execution units under constitutional control, not Chandlerian divisions with independent operating authority.

The agency cost framing of Jensen and Meckling [1976] supplies the residual-loss lens: in recursive AI systems the residual loss dominates because a co-located evaluator can fabricate compliance

signals indistinguishable from genuine progress. The structural prediction is that this failure class is domain-independent; it follows from loop topology (who scores whom), not from substrate, model family, or task domain.

The historical record sharpens the prediction. Chandler-era firms did not self-correct co-location by recognizing its structural logic; external pressure (shareholder litigation, hostile restructuring, and eventually regulatory reporting requirements) was the mechanism that forced the separation between generation and evaluation. That pattern motivates T2: if co-location can persist under optimization pressure until it is externally interrupted, the enforcement floor should not be assumed to emerge from within the loop. It should be specified outside the optimizing process.

3.3 T2: The Hard-Gate Primitive

A hard gate is a governance constraint that is (a) deterministic, (b) fail-closed, and (c) principal-signed. These three properties jointly constitute a governance primitive that prompt-level alignment cannot replicate. Constitutional AI [Bai et al., 2022] governs the output surface: what a model says. The hard-gate primitive governs the enforcement floor: what the system can structurally do.

The institutional precedent for this combination is the external audit profession. The Pujo Committee hearings (1912–1913) documented how concentrated financial self-reporting produced opacity and governance failure in early capital markets; the committee's own report named the core problem exactly: it had "no means of ascertaining the facts so long as their profits are undisclosed." The Securities Exchange Act (1934) responded by interposing a structurally separated verification layer: attestation by a party whose credibility derives precisely from its independence from the operating business it evaluates. The three properties that define the hard-gate primitive map directly onto the audit profession's three controlling institutions: Generally Accepted Accounting Principles supply the *deterministic* rule set, the qualified opinion supplies the *fail-closed* verdict, and the partner signature supplies the *principal-signed* attestation. Chandler gives T1 its architecture (separation of generation from evaluation); the audit profession gives T2 its institutional grounding; it is the historical instance of a hard-gate primitive whose authority derives from its separation from the optimizing loop, not from the capability of the verifier.

The motivation is the invariant: a probabilistic referee cannot satisfy the principal-independence requirement because model output can always soften a probabilistic constraint. T2 is falsified if a system whose enforcement floor contains no deterministic gate can be shown to maintain zero specification-gaming convergence across $k \geq 50$ recursive optimization iterations in a domain where a matched deterministic-gate system exhibits gaming. The criterion is outcome-level: it asks not whether probabilistic enforcement is definitionally possible (it is not, by construction) but whether the absence of deterministic enforcement produces a measurably different failure trajectory under sustained optimization pressure. The empirical evidence that hard gates bind in practice is presented in Section 5.

T3 and T4, developed in the next section, extend the invariant into the observable enforcement floor and the RLHF co-construction analysis.

4 Theory Mechanism

4.1 T3: Observable Enforcement Floor

The principal-independence invariant is necessary but not sufficient. A governance constraint can be deterministic in code and still fail operationally if the principal cannot observe whether the constraint is actually binding. T3 sharpens the theory from existence of a hard gate to observability of the enforcement floor.

In this system, observability means that the principal can inspect the contracted write scope, the append-only event trail, and the deterministic verifier outcome without asking an agent to summarize its own compliance. The hard gate matters because it is outside the optimizing loop; observability matters because it lets the principal tell whether that separation is holding in practice.

The claim is narrower than a general theory of transparency. It does not require full interpretability of model internals. It requires a stable external surface from which the principal can see whether the enforcement floor was triggered, bypassed, or never engaged. A governance primitive that is deterministic but opaque to the principal remains fragile, because the principal cannot distinguish real compliance from theater at the operational layer.

4.2 T4: Co-Construction Extends Co-Location

RLHF-style co-construction does not create a new topology; it extends the original problem to an additional layer. If the policy is optimized against a reward signal that is itself shaped inside the same adaptive loop, the scorer becomes part of the same governance problem. Concretely: if the reward model's training data includes outputs from the policy it scores, the scorer's distribution shifts toward rewarding whatever the policy already produces, closing the co-location loop at the reward layer.

The issue is not that stochastic scorers are uniquely dangerous. The issue is that co-construction can pull the reward model into the adversarial gradient, making the scorer another site where generation and evaluation collapse back together. This connects directly to the reward-hacking literature [Skalse et al., 2022, Casper et al., 2023]: the M-Form's response to reward hacking is structural rather than model-specific. Instead of trying to make the reward signal unhackable, interpose a governance layer that does not use a reward signal at all.

Once co-location reaches the scorer layer, the same remedy applies: the governance constraint must sit outside the co-construction loop. A probabilistic referee inside the loop can still be useful as signal. It cannot constitute the enforcement floor.

4.3 Interface: Why T3 and T4 Do Not Reopen T1 and T2

T3 and T4 extend the foundations argument; they do not replace it. T1 established the structural homology. T2 defined the hard-gate primitive. T3 shows that the primitive must also be observable at the principal layer. T4 shows that RLHF-style co-construction does not escape the theory but can repeat it at a new layer. The result is the paper's central structural claim: in the reported system, the same governance logic appears across layers because the underlying principal-agent problem appears there too.

4.4 T5: Closure Requires an Exogenous Resource Clock

The hard-gate primitive solves *consistency*: the agent cannot generate output and evaluate it inside the same optimization surface. It does not solve *closure*. Closure means knowing when to stop searching, which open question to fund next, and when accumulated negative evidence justifies abandoning a branch. These are not properties an internal architecture can supply, regardless of how strict the gates are.

Empirical evidence comes from a parallel substrate run on which the same recursive system was tasked with discovering the optimal organizational topology for a generic agent–human firm. Across thirty-eight evaluations, a bicameral / dual-confirmation thesis family attracted scores in the seventy-nine to ninety-three band repeatedly, with each peak cut down on the same controlling assumption: the audit-path independence axiom. The most recent run was reverted to a final score of twenty with the verdict that the thesis "asserts eternal openness rather than defining when or how to focus resources, making it simultaneously untestable and paralyzing." That reversion is itself the result. Architectural consistency does not provide a stopping rule.

Three properties turn out to be the same property under different names: when to stop searching, which open question to fund next, when accumulated negative evidence justifies abandoning a branch. None of them is supplied by the OS layer or the Config layer. They require an exogenous resource clock: a budget, a deadline, an investor's quarterly cycle, a mortality timeline, an attention budget. Without that clock, an M-Form architecture run on infinite resources correctly converges on epistemically honest but operationally paralyzed theses. The principal in the implementation reported here is not a placeholder for governance legitimacy; the principal *is* the exogenous clock.

This sharpens the M-Form claim. T5 names a third structural primitive alongside the principal-independence invariant (T1, T2) and the observable enforcement floor (T3): the **exogenous resource clock**. The full M-Form description is layers plus clock. Classical accounts often treat the clock as an environmental given because the firm exists in capital markets that allocate quarterly; recursive AI governance must specify the clock explicitly because the system can be run on effectively infinite synthetic compute and lose closure entirely. T5 closes the loop opened by T1: T1 said co-location of generation and evaluation produces gaming, T2 defined the deterministic enforcement floor, T3 made it observable, T4 showed co-construction does not escape the topology, T5 says the architecture stabilizes only against an exogenous clock; without one the M-Form is consistent but paralyzed, a failure mode functionally equivalent to gaming.

4.5 Formal Model: Effective Capability Under Co-Location Pressure

The architectural argument of Sections 3–4 admits a small formal model that consolidates T1–T5 into testable propositions. Define five state variables. $C \geq 0$ is the raw capability of the underlying agent — the axis the Bitter Lesson scales. $\varepsilon \in [0, 1]$ is the error rate, the fraction of iterations producing genuine errors detectable by external test. $g \in [0, 1]$ is the gaming rate, the fraction of iterations that pass internal evaluation while violating its controlling intent. $n \geq 1$ is recursive loop depth. $\pi \in [0, 1]$ is principal attention, where 1 is continuously present and 0 is fully absent.

Effective capability in a recursive loop is

$$E(C, g, \varepsilon, n, \pi) = C - A(g, \varepsilon, n, \pi) \tag{1}$$

where A is the agency cost. Errors compound linearly with depth; gaming compounds super-linearly

and is more visible to a present principal:

$$A(g, \varepsilon, n, \pi) = \alpha \cdot \varepsilon \cdot n + \beta \cdot g \cdot n \cdot (2 - \pi) \tag{2}$$

with $\alpha, \beta > 0$ the layer-specific costs of error and gaming. The crucial property is that the gaming term scales with $(2 - \pi)$: when the principal is fully present ($\pi = 1$), gaming cost is $\beta g n$; when absent ($\pi = 0$), gaming cost doubles because no immediate reality check intervenes.

The five propositions follow from the model.

P1 (Co-location induces gaming). In a unitary-form architecture, where generation and evaluation share the same optimization surface, $g_U > 0$ and increases with n under sustained optimization pressure. This is the Section 5.4 recurrence claim, formalized.

P2 (M-Form bounds gaming). A deterministic enforcement floor with no learned parameters reduces g to approximately zero *for the property the floor checks*, regardless of n. Gaming continues for properties not checked by the floor; the M-Form's enforcement boundary is exactly the set of properties for which $g_M \approx 0$ holds.

P3 (Crossover). Let $h \geq 0$ be the fixed per-cycle overhead of the M-Form. The M-Form's effective capability is $E_M = C - \alpha\varepsilon n - \beta g_M n(2-\pi) - hn$; the unitary form's is $E_U = C - \alpha\varepsilon n - \beta g_U n(2-\pi)$. The M-Form dominates the unitary form when

$$\beta \cdot (g_U - g_M) \cdot (2 - \pi) > h. \tag{3}$$

Since P2 sets $g_M \approx 0$ and P1 sets $g_U > 0$ increasing with optimization pressure, the M-Form pays for itself when $\beta \cdot g_U \cdot (2 - \pi) > h$. The gaming-suppression value $\beta \cdot g_U$ scaled by principal-absence $(2 - \pi)$ must exceed the fixed overhead h. When $\pi = 1$ sustained, the M-Form's overhead is harder to justify; when $\pi = 0$, the gaming-suppression value is doubled.

P4 (Exogenous clock requirement, restating T5). Let γ be the closure rate. Without an exogenous resource clock, $\gamma \to 0$ as $n \to \infty$, regardless of architecture. The M-Form alone is consistent but paralyzed; closure requires a clock external to the optimizing loop. Examples of admissible clocks include capital-allocation cycles, regulatory-audit cycles, mortality timelines, and the principal's finite attention budget.

P5 (Self-reference under optimization pressure). A recursive evaluation system that uses only signals it produces to score its own evolution cannot fully evaluate its own falsification machinery from within its own scoring frame. The loop can identify candidate-proposal failures but cannot reliably distinguish "evaluator detecting failure" from "evaluator's own definition of failure has drifted under optimization pressure." This is the Appendix A.3 finding stated as a structural property of self-referential evaluation systems, consistent with Tarski's hierarchy result and Gödel's incompleteness in the philosophical literature.

Boundary conditions. The model assumes (i) ε is independent of g — gaming and errors are different failure modes; (ii) the principal's attention budget is finite, so $\pi \cdot n$ is bounded and $\pi = 1$ sustained over large n is not feasible; (iii) the per-cycle overhead h does not scale with C — the M-Form's overhead is set by the contract complexity, not by the agent's capability. Where these assumptions break, the model needs extension. P1–P5 hold within these conditions; the calibration of α, β, h, and the empirical relationship between g and (n, π) are open questions this paper does not resolve.

The formalization makes Section 6.4's "effective capability" argument testable. A controlled comparison would measure g_U and g_M for the same task family and the same agent capability C, vary n and π, and check whether the crossover condition predicts the result. The single-implementation evidence in Section 5 is consistent with the model but does not calibrate it.

5 Empirical Evidence

The evidence in this section is drawn from one system: a deterministic supervisor controlling multiple agent programs over a four-month development period, implementing the OS / Config / App decomposition introduced in Section 2. The primary case study is a manuscript-production program that executed 38 state transitions over approximately 5.5 hours, producing one theory section through the full pipeline at a total cost of $1.36 (plus $0.71 in specification refinement). Additional evidence is drawn from the evaluator layer (nine documented specification gaming strategies across five domains), the kernel hardening layer (typed derivation failures), and the blind-protocol drafting layer (context contamination under shared context windows). All claims are scoped to this system; generalization requires independent replication.

5.1 Enforced Governance

The write-scope guard is a Python function that snapshots the repository before and after each agent invocation. Any file modification outside the contracted artifact set triggers a fail-closed transition to a human-review state. The agent cannot suppress, delay, or negotiate this enforcement; it is a post-hoc diff comparison that the agent does not control.

The mechanism is regression-covered by the fixture suite (18 of 18 tests passing), which includes an explicit unauthorized-write case: an agent writes to a path outside its contracted scope; the wrapper detects the modification; the staged request is marked as a scope violation; the state machine routes to the human gate.

In live operation, the write-scope guard fired twice during the manuscript program's first run. The specification-writing agent (an OpenAI o4-mini instance operating as the skeptic) was contracted to produce a prose specification for the manuscript opening. Instead, it modified four files outside its contracted scope: the project build file, the autoloop orchestration module, the autoloop fixture regression, and the supervisor wrappers. The guard detected the unauthorized modifications via repository diff, flagged the scope violation, and routed to the human gate. The artifacts (including the full agent prompt, stdout, stderr, and usage telemetry) were archived. The run was restored to the last committed state. The same agent violated the boundary again on the next cycle (same pattern plus an output-token-limit breach). In a subsequent run with a narrower contracted scope (one markdown file and one JSON specification), no unauthorized write was attempted.

5.2 Constrained Self-Hosting

The system modified its own governance surface through its own governance protocol without dissolving that protocol. Over 92 debate turns, the modifications included:

- **Verification extension.** The supervisor had no verification contract for prose output. Three model families (Claude, Gemini, GPT-4o) debated the fix through the governed protocol. The result was a deterministic prose verifier and a typed assertion schema: a new verification capability added through the same governed debate that produced all prior code-verification improvements.

- **Transport hardening.** The CLI-to-API migration was discovered through cost telemetry (the CLI path imposed a ∼340,000-token context floor per invocation), debated, implemented, and regression-tested within the governed loop.
- **Specification-writer constraint.** The specification-writing agent was discovered to be generating cosmetic assertions (word counts, capitalization checks) instead of assertions testing controlling theoretical claims. The failure was typed, a hard prompt constraint was inserted banning cosmetic specifications, and the regression suite confirmed the constraint holds.

In each case, the governance protocol (debate, implement, verify, gate) remained intact while the governance surface changed. The system did not merely repair defects; it ratcheted named failure classes into constraints that narrow the optimization surface for future agents. Each typed failure class (unauthorized write, specification gaming, cosmetic verification) is now a permanent fixture-covered enforcement rule. This is distinct from ordinary iterative development in that the governance layer itself (not just the application code) was the subject of improvement, and the improvement was constrained by the same deterministic enforcement floor it modified.

5.3 Operator Surface

The manuscript program produced a structured management surface: a machine-readable status file, an append-only event trace (39 entries), a deterministic verification report, and a debate file (37 turns of architectural provenance). The principal managed 38 state transitions and resolved four human gates (three specification-refinement caps, one contract promotion) using these artifacts rather than raw conversational context.

The evidence supports a scoped claim: a single principal can manage one active program with structured summaries and typed gates. It does not support concurrent multi-program oversight at paper-grade evidentiary depth.

5.4 Recurrence of Specification Gaming

The same Goodhart pattern (the optimizer satisfies the letter of the specification while violating its spirit) was independently observed at five layers of the system. The evaluator, kernel, and apparatus-specification findings are reported fully in prior work and cited here; the supervisor and interface-contract findings are this paper's contribution.

Layer 1: Evaluator. Alami [2026a] documents nine specification gaming strategies across five domains of adversarial evaluation, all self-certifying: generated code passes its own assertions while violating the epistemic intent of the evaluation rubric. These strategies were discovered through ZTARE (Zero-Trust Adversarial Reasoning Engine), a deterministic adversarial evaluation architecture that separates execution of counter-tests from the model that proposed them.

Layer 2: Kernel. Alami [2026b,c] document specification gaming at the evaluator kernel layer: the derivation gate was found to be gameable through label hardcoding, allowing fabricated theses to reach the aligned-evidence path instead of failing closed. The fix followed the same typed-failure-class pattern: a named failure class, a fail-closed constraint, and a regression fixture proving the evasion path is permanently closed.

Layer 3: Supervisor. In this system, the specification-writing agent generated assertions targeting cosmetic properties (word counts, capitalization, formatting) instead of controlling theoretical claims. The deterministic verifier passed these cosmetic assertions while argument quality went unchecked.

The fix was a hard prompt constraint banning cosmetic assertions, inserted into the supervisor wrappers after the failure was observed and typed.

Layer 4: Apparatus specification. During the design of a physics-function recovery sandbox, a pre-commit bootstrap verifier was built to check whether the operator-authored ground-truth function was uniquely identifiable from the visible data before any mutator run was permitted. The verifier passed: one hundred bootstrap refits returned parameter standard deviations well below the gate threshold, indicating stable identification. But the verifier was checking the wrong property. Parameter stability under resampling does not detect whether two parameters have algebraically collapsed into their ratio throughout the functional form: a structurally distinct identifiability failure. The ground-truth family contained two parameters α and β that appeared throughout exclusively as the compound $\gamma = \alpha/\beta$; the bootstrap fitter converged on the ratio without ever perturbing α and β independently, and the verifier reported stable identification of a five-parameter family when the family was in fact rank-four. The pre-commit check passed, the sandbox was provisionally sealed, and the degeneracy was caught only by adversarial multi-start resampling after the verifier had already approved the packet. The fix required a symbolic algebraic-combination audit (checking, before any run, whether any sub-tuple of parameters could be compressed into a single compound) and a reparameterization that made the compound explicit before the mutator saw the sandbox. The pattern is structurally identical to Layers 1–3: the pre-commit verifier satisfied the verifiable surface (bootstrap consistency) while evading the controlling intent (detecting reparameterization degeneracy). The substrate here is a bootstrap script rather than a language model, but the Goodhart topology is unchanged.

Layer 5: Interface contract. A deterministic evaluation harness was built to score proposed mathematical models by importing a callable function `f(u, v)` from the model file and testing it against held-out data. A separate prompt instructed the generating agent to write that model file. The prompt, written for a different evaluation mode, told the agent to produce assertion-based test code. The harness expected a callable function definition. The agent did exactly what the prompt asked: it wrote well-formed assertion suites, every iteration, for five consecutive iterations. The harness crashed every iteration because the callable it tried to import did not exist. Neither the agent nor the harness malfunctioned; each satisfied its own contract. The failure lived at the interface: the prompt contract and the harness contract specified different artifacts, and the agent optimized against the prompt surface because that was the specification it could see. The fix was to add the harness's contract to the prompt, so both contracts were visible to the agent before generation. The pattern is the same as Layers 1–4: the specification surface was satisfied while the controlling intent (producing a scorable model) was not. The substrate is the interface between two deterministic contracts, not a model or a script; the topology is unchanged.

The convergence is structural: at each of five layers, the agent or process under optimization pressure found the path of least resistance through the specification surface. The specification differs at each layer (evaluation rubric, derivation gate, prose specification, pre-commit bootstrap rule, interface contract). The gaming strategy is structurally identical: satisfy the verifiable surface while evading the controlling intent. Sections 5.7 and 5.8 report sixth and seventh instances at, respectively, the human-plus-warm-agent drafting layer and the blind-protocol drafting layer.

Independence structure and honest scope of the recurrence claim. Four of the seven instances are documented within this paper: Layers 4–5 (apparatus specification, interface contract) were observed during sandbox construction for the present system, and Layers 6–7 (warm-agent drafting, blind-protocol drafting) were observed in live sessions during the writing of this paper. The remaining three instances (Layers 1–3, at the evaluator, kernel, and supervisor layers) are

background motivation drawn from the author's prior working papers [Alami, 2026a,b,c]. Those companion papers are cited as motivation for taking the pattern seriously enough to look for it across additional layers; they are not asserted here as independent confirmations of the present argument. The honest claim is therefore four observation clusters in this paper plus three earlier instances from related work by the same author across different substrates. All seven observations share an investigator and overlap in apparatus, which limits any claim of independent confirmation. The four primary observations are separated by substrate (deterministic scripts and contract interfaces in Layers 4–5; human-agent co-authoring in Layers 6–7) and by temporal phase, which is the structural property the recurrence argument actually rests on: the adversarial gradient appears at distinct layers under sustained optimization pressure, consistent with T1's domain-independence prediction. We do not claim the four (or seven) observations together establish the pattern's general applicability beyond the reported system; that requires independent replication and is named as out of scope in Section 1 and Section 7.

5.5 Excluded Claim: Capital Efficiency

The system does not yet support a rigorous bounded-versus-unbounded cost comparison. The $1.36 program cost and $0.71 refinement cost for one manuscript section provide a data point but not a controlled comparison. Capital efficiency is a relevant question but not yet an evidentiary claim.

Since the initial manuscript run, the system has accumulated per-iteration machine telemetry across 22 completed runs spanning 251 governed iterations at a total compute cost of $13.39. The mean cost per governed iteration is $0.05 with a mean wall-clock time of 205 seconds per iteration. Runs range from $0.09 (5-iteration domain analysis) to $2.69 (15-iteration geopolitical scenario with Gemini Pro), with physics-substrate runs averaging $0.02–$0.11 per iteration depending on model choice and substrate complexity. The telemetry infrastructure (GP-038/GP-040 shared iteration stream) now records per-iteration cost, wall-clock timing, model usage, gate engagement, and stop reason for every governed iteration.

These numbers establish the cost floor of governed recursion but do not yet constitute a capital-efficiency claim. The missing comparator is the same task executed without governance: an ungoverned multi-step loop on the same substrate, with the same model, where agency cost (fabricated compliance, context corruption, rework) can compound unchecked. Until that controlled comparison exists, the aggregate telemetry is a denominator without a numerator. The infrastructure to run the comparison is in place; the comparison itself is deferred to future work.

5.6 Build Pipeline Evidence

The prose pipeline telemetry in Section 5.3 is a negative-ROI case: the M-Form's overhead dominates the cost structure when the work is a single document and the principal is attentive. The build pipeline is the predicted positive-ROI case: a code artifact with deterministic verification criteria, a declared write-scope boundary, and execution that the principal does not need to watch line by line. This subsection reports telemetry from running that case through the same supervisor stack, so that the two cases can be compared against the same instrument.

Program and packet. The reference run is a closed governance program whose contracted task was to wire a state-derivation seam evaluator into the live upstream handoff path as a kernel-facing fail-closed gate. The packet preserves the frozen downstream contracts of the later pipeline stages while blocking fabricated safe-harbor anchors at the upstream boundary. The contract is a real

kernel modification, not a toy benchmark: four source files are declared in advance as the authorized write-scope, and any write outside that set must fail closed.

Declared write-scope. The packet manifest names exactly four authorized artifacts: the seam evaluator module, a handoff wiring module, and two fixture-regression suites. At closure, the run's implementation snapshot records sha256 fingerprints for exactly these four paths and no others. The authorized set and the modified set match exactly; no file outside the manifest was written during the run. This match is the file-level realization of the write-scope guard described in Section 5.1; the guarantee is recorded in the run's closure record, not asserted in prose.

Verifier contract. The packet specifies a deterministic verification command consisting of three regression suites run as a shell chain: a state-derivation fixture suite, a cross-stage bridge fixture suite, and a downstream-stage fixture suite. Each suite asserts exact classification labels against frozen input fixtures. There are no probabilistic judgments in the verification chain. A surviving packet must pass every fixture case or the verifier fails closed.

Cost ledger. The run's final status records a program cost of $0.6463, a refinement cost of $0.0000, zero spec-refinement rounds, and zero consecutive build failures. Zero refinement rounds means the first spec emitted by the specification-writing agent was accepted by the verifier on the first build pass. Zero build failures means no fail-closed event fired during the packet execution. The refinement ledger is distinct from the program ledger in this system (Section 5.3 reports both as $1.36 and $0.71 for the prose case); the build pipeline's refinement ledger was never touched.

State transitions and actor distribution. The closing revision's event stream records five transitions from first actor entry to human closure: a specification turn, a registration turn acknowledging pre-existing work, an implementation turn, a deterministic verifier turn, and a human gate resolution. The actors are structurally separated by role: one model family specifies, a second model family implements, a deterministic non-model verifier checks, and the human principal sits at the gate. No single actor held more than one role in the sequence. The final transition passes under a gate-on-verifier-pass policy with the human-gate reason recorded as a contract promotion and the human gate resolved.

Verifier results, reproduced for this section. The verification command was re-run after closure against the sha256-snapshotted implementation set recorded in the run's closure file. The three suites returned 4/4 cases on the state-derivation fixture suite, 8/8 cases on the cross-stage bridge suite plus 3/3 live upstream-gate smoke tests on markdown surfaces, and 8/8 cases on the downstream-stage suite. Total: 23/23 deterministic fixture cases pass, reproducing the closure-time verdict three days after the run closed, against frozen code snapshots. This reproducibility is the structural property the paper claims for deterministic enforcement: a verifier whose output is not a function of model temperature, sampling, or context window state returns the same verdict on the same inputs regardless of when it is run.

Completion state. The program registry shows the run as closed with owner mode frozen after the human gate resolved. The manifest's three packets all show status complete. No packet required a re-execution loop.

Comparison with Section 5.3. The prose pipeline in Section 5.3 produced a single manuscript section at a program cost of $1.36 and a refinement cost of $0.71, with the principal actively reviewing nearly every transition. The build pipeline here produced a kernel-level seam hardening at a program cost of $0.65, a refinement cost of $0.00, zero refinement rounds, and zero build failures, with the principal reviewing only the final human gate. The cost reduction is not the

interesting number; the structural difference is that the build pipeline's verifier returned a hard pass/fail verdict on every check, while the prose pipeline's verifier could only check deterministic conformance (headers, citations, word counts) and the semantic quality of the prose still depended on principal attention. The crossover hypothesis Section 6.4 names (that the M-Form's overhead structure has positive ROI precisely when the work has a deterministic verifier and a declared write-scope) is not proven by a single run. But it is now empirical rather than hypothetical: the two reference points exist on the same instrument and can be pointed at directly.

Honest scope. This is one program, closed. The run is not a controlled experiment and it is not a generalization claim. The cost ledger is a single data point; a rigorous capital-efficiency comparison remains the Section 5.5 excluded claim. What this subsection demonstrates is narrower and more concrete: the M-Form supervisor ran a real kernel modification to completion through its normal state machine, the verifier contract held at closure and reproduces today against the same frozen code, the write-scope enforcement matched declared intent exactly, and the refinement budget was never needed. These are the properties Section 4's theory predicts for the build-pipeline case, and they are now recorded with file-level provenance rather than argued in prose. The full audit trail (the run's final status record, the event stream, the verification report, and reproduction instructions) is included in the paper's replication package (see Data and Code Availability).

Additional observations. Four further observations of the same Goodhart pattern documented in Section 5.4 are reported in Appendix A: a live catch in a catch-grammar self-audit, a context-contamination incident in a blind-protocol experiment, a self-reference observation from a meta-recursive substrate test, and a methodological adjacency note on the panel-of-roles substitute for substrate-mode evaluation. Those observations are presented as additional cases consistent with the framework rather than as load-bearing tests of it; the load-bearing observation is Section 5.4's narrowed recurrence claim. Readers focused on the theoretical contribution may proceed directly to Section 6.

6 Counterarguments and Boundaries

6.1 The Chandler Convergence Is Structural, Not Anthropomorphic

The Chandlerian analogy operates as a convergence claim about structure rather than a stylistic metaphor. Human firms and recursive AI systems both encounter governance failure when generation and evaluation are co-located under optimization pressure. The common solution is separation of roles: not because firms and models are secretly the same thing, but because the underlying control problem has the same shape. The analogy is used where it is controlling (co-location, scope separation, rate-of-change separation) and stopped where it is not (divisional autonomy, market allocation, human organizational culture).

6.2 The Bitter Lesson Does Not Solve Governance

The Bitter Lesson [Sutton, 2019] is a claim about representation learning: methods that leverage computation scale better than methods that leverage human knowledge. It is not a governance theorem. Better learned representations can improve capability and still worsen camouflage; a stronger model can become better at manufacturing compliance theater. The relevant distinction is whether the enforcement floor sits inside the optimizing loop or outside it, regardless of whether either is learned or hand-engineered. A deterministic governance layer with no learned parameters operates as a structural constraint on the optimization surface rather than as a hand-engineered

feature in the Bitter Lesson's sense.

6.3 "This Is Just Good Software Engineering"

The most likely objection to the self-hosting evidence (Section 5.2) and the recurrence evidence (Section 5.4) is that they describe ordinary iterative development with tests. Every CI/CD system catches unauthorized writes. Every code review catches drift. The response requires precision about what the M-Form adds beyond standard practice.

The objection has real force. Much of what the M-Form does (write-scope enforcement, deterministic assertions, pre-commit guards) is bread-and-butter software engineering. The claim is not that these techniques are novel in isolation. The claim is that *their application to the evaluator itself, under recursive optimization pressure, is a structurally different problem than their application to production code*. Three distinctions matter.

First, *co-location of authorship*. In standard software engineering, the test suite is authored by the same team, often the same person, that writes the code. A developer who writes both implementation and tests can write tests that are guaranteed to pass. CI/CD catches failures against the spec; it does not catch whether the spec itself was gamed. The M-Form's structural contribution is that the governance layer has no learned parameters and cannot be negotiated by any agent in the system, including agents that propose changes to it. The write-scope guard does not ask the agent whether it stayed in scope; it computes the diff.

Second, *adversarial pressure on the evaluation surface*. In standard development, the code is not trying to satisfy the test suite in the game-theoretic sense; the developer wants the tests to be honest because the developer wants working software. In recursive AI optimization, the agent *is* trying to satisfy the evaluation contract, and its optimization pressure acts directly on the evaluation surface. Every specification gap is a potential basin of attraction. This is why specification gaming is not a bug to be patched one-off but a structural feature of the loop: the agent's gradient always points toward satisfying the letter of the contract, and any gap between letter and intent is exploitable under sustained optimization. Standard CI/CD assumes good-faith authorship; the M-Form assumes adversarial optimization pressure on the evaluation contract itself.

Third, *recursive depth*. Standard software engineering is typically flat: code is written, tested, and deployed. The M-Form governs a recursive loop where the output of iteration n becomes the input to iteration $n+1$, and agency cost compounds with loop depth. A one-off test failure is caught and fixed. A specification gaming strategy that evolves across iterations (the kind documented in Alami [2026a]) behaves as a convergent pattern that passes every test while violating the test's intent, rather than as a single-test failure. The enforcement floor must be deeper than any individual test: it must be structural, deterministic, and non-negotiable across the full iteration history.

6.4 Overhead Versus Raw Capability

The M-Form imposes overhead. That is a real tradeoff, not a rhetorical concession. The overhead is a fixed cost: genesis artifacts, write-scope enforcement, deterministic verification, and human gates at state boundaries. The agency cost of ungoverned recursion is a variable cost that compounds with loop depth; each iteration that fabricates compliance rather than producing genuine progress wastes the tokens spent on that iteration and corrupts the context for subsequent iterations.

The paper's argument is that effective capability is raw capability discounted by agency cost. A system with higher nominal autonomy but weak governance can underperform a more constrained

system once self-deception and rework are priced in. The crossover scales with loop depth multiplied by principal absence, not loop depth alone. This is the same economic boundary that drove the historical transition from founder-run firms to the corporate M-Form: Chandler's general office was not invented for founders who were in the room watching every transaction. It was invented for absentee shareholders who needed to coordinate thousands of employees across geographies without going bankrupt from agency costs. When the principal is present and attentive in every turn, governance overhead is pure drag, paying the cost of absentee governance while being hyper-present. The M-Form pipeline's 38 state transitions and $2.07 to produce one manuscript section over roughly 5.5 hours would be strictly dominated in that regime by a lighter-weight protocol in which the same principal interacts directly with two agents through a shared document, for the simple reason that the principal is already supplying the separation the M-Form exists to enforce. When the principal is absent or managing concurrent workstreams, the governance overhead is the only thing preventing agency cost from compounding unchecked. The crossover point (the combination of loop depth and principal absence at which governance overhead is dominated by the agency cost it prevents) is a testable hypothesis that the current system does not yet establish empirically, and the paper does not attempt to pin it to a specific cost or wall-clock threshold from a single data point.

6.5 Principal Scale

The current evidence is for a single principal supervising one recursive system. The architecture may generalize; the evidence does not yet prove multi-principal or large-organization operation. That boundary is a strength if stated clearly and a weakness if blurred.

7 Related Work and Limitations

7.1 Constitutional AI and RLHF

Constitutional AI [Bai et al., 2022] and RLHF [Ouyang et al., 2022] constrain behavior through prompts, reward shaping, and learned preference models. They operate on the output surface: what the model says. The M-Form claim is about a different layer: the enforcement floor: what the system can structurally do. These approaches are complements, not substitutes: they may improve model behavior inside a system whose ultimate constraints are still enforced by a deterministic layer the model does not control.

7.2 Multi-Agent Frameworks

Multi-agent orchestration frameworks (exemplified by AutoGen [Wu et al., 2023] and similar systems) separate agent roles conversationally: one agent generates, another critiques, a coordinator routes. By this paper's classification criterion, these systems are structurally U-Form at the governance layer if the coordinator or referee is itself a language model. Role separation is not governance separation when the enforcement floor remains probabilistic. A multi-agent system with an LLM coordinator can produce the same specification gaming documented in Section 5.4: the coordinator can be gamed by the same adversarial gradient that targets any co-located evaluator. Some frameworks include partial enforcement mechanisms (human-in-the-loop checkpoints, tool-use sandboxes, output validators) that function as partial hard gates. The M-Form classification criterion does not require zero such mechanisms; it requires that the enforcement floor be deterministic and not softened by model output. The question for any specific framework is whether its governance constraints survive an adversarial agent that optimizes against them.

A sharper formulation of the U-Form failure in multi-agent frameworks comes from examining which tasks are routed to the LLM. In the dominant paradigm [Yao et al., 2023, Yang et al., 2024, Richards et al., 2023], the LLM simultaneously serves as router, diagnostician, and executor: it decides what state the system is in, what action to take, and then executes that action. This triple co-location is the Chandlerian pathology at the task level: one process generating output, evaluating it, and deciding on the basis of its own evaluation. Chandler's (1962) remedy was structural separation: a general office that performed resource allocation and performance monitoring using objective data, separate from the divisions being evaluated. The M-Form correction in AI systems follows the same structure. State diagnosis (determining whether the system is stagnant, thrashing, or resource-exhausted) is computable from telemetry and belongs to the deterministic layer. Hypothesis generation (deciding what to try next) is not computable from telemetry and belongs to the probabilistic layer. When the system can classify its own state by computation, the LLM's role is confined to the task for which no deterministic substitute exists: proposing what to try next. This is why the described system does not loop endlessly. The enforcement floor is deterministic, so the system identifies structural paralysis through computation rather than by asking the LLM to interpret its own failure.

7.3 Differentiation, Integration, and Task Interdependence

The M-Form architecture draws on Chandler (1962) for the separation principle and Jensen and Meckling (1976) for the agency cost lens, but two foundational contributions to organizational design theory sharpen the structural claims. Lawrence and Lorsch [1967] established that effective organizations must simultaneously *differentiate* (create specialized units adapted to distinct environmental segments) and *integrate* (coordinate those differentiated units toward a common purpose). The deterministic gate is the M-Form's integration mechanism: it does not require the differentiated layers to share context, training, or optimization objectives. Thompson [1967] classified task interdependence into three types — pooled, sequential, and reciprocal — that map directly onto the M-Form's internal structure; reciprocal interdependence between mutator and judge is converted into a managed sequential flow with a deterministic feedback arc by routing the failure diagnostic through a structured JSON constraint rather than free-form negotiation. The recent literature on meta-organizations (Ahrne and Brunsson, 2008; Berkowitz and Dumez, 2016) extends Chandler and Thompson to organizations whose members are themselves organizations; Appendix B's institutional sketch is a meta-organizational design.

7.4 Evolutionary Economics, Capabilities, and Recent ICC Conversation

The OS / Config / App decomposition is a capabilities architecture for an AI-operating firm in the sense Nelson and Winter [1982] gave that term: a system of routines whose execution under selection pressure produces the firm's characteristic outputs. The M-Form's hard gates are the firm's *deterministic routines* — patterns of action that are not negotiated each time they fire and that survive personnel turnover precisely because they do not depend on individual judgment. Dosi [1982, 1988] frames technological change as the cumulative articulation of paradigms within which trajectories evolve under selection; the recurrence of specification-gaming patterns documented in Section 5.4 is, on that reading, the same selection pressure operating at the AI-evaluator layer that Nelson and Winter described at the firm-routine layer. The dynamic-capabilities literature [Teece et al., 1997] extends this to the *recombination* of routines under environmental change; the strange-loop test reported in Appendix A.3 is an early observation of a recursive firm attempting routine recombination on its own evaluation machinery, and the failure mode it surfaces (self-reference under optimization pressure, P5) is a structural limit on recombination from inside the system.

The contribution of this paper relative to that tradition is narrower than the tradition itself but specific to a new class of firms. Nelson and Winter's framework was built for firms whose routines are executed by humans following rules-of-thumb under bounded rationality; the present paper extends the framework to firms whose routines are executed by language models under sustained optimization pressure, where the selection environment now includes the routine's own evaluation function. The gaming-rate variable g in Section 4.5 is the cost of letting the optimization pressure act on the evaluation routine; the M-Form's hard gates are the structural response.

Recent contributions to this journal sharpen the empirical horizon. Schrepel and Pentland [2025] document concentration dynamics among AI foundation models and identify the policy levers by which a few platforms could fix the AI ecosystem's evolutionary trajectory; the present paper's contribution is orthogonal to but compatible with that line: the M-Form is a *firm-internal* governance architecture, while Schrepel and Pentland address the *between-firm* competitive structure under which such firms operate. A 2025 ICC advance article on measuring artificial intelligence and its governance implications [Hötte et al., 2025] examines firm-level AI development concentration; the M-Form architecture proposed here is one candidate response to the governance gap that line of work documents at the population level. Adjacent recent work in *Journal of Management Studies* [Glaser et al., 2024] advances "organizations as algorithms" as a metaphor for advancing management theory; this paper's contribution is the inverse direction, arguing that the M-Form is the architecture under which AI agents operating inside firms produce coordinated output without dissolving into the recurring-Goodhart pathology Section 5.4 documents.

7.5 Process Supervision

Process reward models [Lightman et al., 2023] score intermediate reasoning steps rather than only final outputs, improving evaluation granularity. This is the closest existing work to the M-Form's step-by-step governance. The distinction: process reward models use a learned scorer; the scoring model's parameters are optimized, and the model's outputs can drift under distributional shift. The M-Form's verifier is deterministic Python with no learned parameters. A process reward model improves signal quality; it does not guarantee a stable enforcement floor.

7.6 Institutional Verification Traditions

The closest institutional analog to the M-Form's hard-gate primitive is not a technical system but an organizational one: the external audit profession. Independent audit interposes a structurally separated verification layer between an operating enterprise and its principal (shareholders, regulators, counterparties). The verifier's credibility is a function of three properties: rule-boundedness (the audit follows a published methodology not set by the audited entity), independence (the auditor has no operating stake in the outcome), and liability (attestation creates accountability if the verification is wrong). These properties map onto the M-Form's governance primitives in partial form: the deterministic gate is rule-bound by the principal-signed contract, structurally separated from the agent's optimization loop, and fail-closed rather than advisory. The liability component is not yet instantiated and remains a limitation.

This framing sharpens the contrast with Constitutional AI (Section 7.1). Constitutional AI is a *deontological* primitive: the model is trained to hold values and apply them to its own output. The audit profession is an *institutional* primitive: external rule-bound verifiers with authority that does not depend on the verifier sharing the agent's values or training distribution. T2 adapts the institutional-verification logic to recursive AI, while leaving the legal and liability parts of the

analogy unresolved.

The audit profession also supplies the failure case. Arthur Andersen's collapse following the Enron audit illustrates what happens when the verification function is co-located with advisory and consulting revenue: the verifier acquires a stake in the outcome, the independence property erodes, and the attestation loses its credibility function. The M-Form addresses the structural root of that failure, not by making the verifier more capable, but by keeping it outside the optimizing loop.

This framing also distinguishes the M-Form contribution from process supervision (Section 7.5). Process reward models improve signal quality; they do not address independence or liability. The institutional verification literature (audit and regulatory attestation) is the relevant comparison class for the governance claim.

7.7 Limitations

This paper rests on one system operated by one principal over four months. That is the central limitation and it should stay in the foreground. Additional limitations: (a) the system has been tested with three model families (Claude, Gemini, GPT-4o) but only in one implementation; the same architecture with different codebases has not been tested; (b) the principal is a single domain expert; the architecture has not been tested with a non-expert principal or with multiple concurrent principals; (c) the prose pipeline's factory overhead was negative ROI for a single document; the same manuscript was later produced through a manual debate protocol in a fraction of the time and cost, suggesting the M-Form's cost structure favors high-volume, high-stakes, or principal-absent production over artisan work with an attentive principal; (d) prose quality and operator ergonomics remain weak points; the architecture has been proven more strongly than the user experience; (e) recent evidence of subliminal trait transmission during model distillation (Cloud et al. 2026) demonstrates that behavioural traits can transfer through semantically unrelated training data when teacher and student share the same base initialization. The mechanism operates during fine-tuning, not during inference-time in-context reading, so the direct threat to the M-Form's Division A/B protocol is limited. The upstream concern (that the model families themselves may carry subliminal traits inherited during their own training pipelines) is real but outside the M-Form's architectural scope. Cross-model diversity partially mitigates this, since Cloud et al. [2026] report the effect requires shared initialization and is weak across different architectures.

A structural gap in the audit analogy is the absence of a liability framework. Independent audit derives much of its credibility from the fact that a wrong attestation creates legal and reputational exposure for the auditor. The M-Form's hard-gate primitive is fail-closed and deterministic, but the verifier bears no liability for a false pass: if a gate fires incorrectly, or if the gate design is under-constrained, the principal absorbs the cost. Three candidate mechanisms could supply a liability-adjacent property: the operator as accountable party (the audit-partner analog), gate-library reputation (a reputation primitive weaker than legal liability but real), or an internalized-no-external-consumer model in which verification is itself the product consumed by the same principal who owns the underlying decision. The audit analogy is incomplete without one of these, and any broader external-facing verification regime will likely need one; Appendix B's institutional verification sketch is one place to look for candidate solutions.

7.8 Pending Evidence and Future Work

Evidence base status. The empirical evidence base for this paper is populated. Section 5.6 reports telemetry from a closed build-pipeline run against frozen sha256-snapshotted implementation files,

and the deterministic verifier chain reproduces the closure-time verdict on re-run. Appendix A.1 reports the live catch grammar self-audit with file-level artifacts already on disk. Appendix A.2 reports the context contamination incident from a blind-protocol experiment, demonstrating the OS-layer context isolation fix in the goal orchestrator. The three subsections report different classes of evidence for different claims and all are required for the paper's argument. The Rule 1 Auditor Prompt spec's mandatory discrimination test gate remains an explicit non-claim: it is not asserted by the paper as written, and it must not be wired into a live run until that gate passes.

Post-publication research agenda. The immediate external agenda is replication on other systems, stronger capital-efficiency measurement, a cleaner principal abstraction layer, and empirical testing of the subliminal learning channel identified by Cloud et al. [2026], specifically whether the training-time transmission mechanism they demonstrate has any inference-time analog in the M-Form's Division A/B hand-off protocol, and whether cross-model diversity is a sufficient architectural mitigation for the upstream training-pipeline concern. A more ambitious line is governance self-hosting under rule-bound independence: agents proposing changes to the governance contract while remaining subject to a deterministic check that the revised contract preserves the principal-independence invariant. A nearer-term empirical direction is signal-guided structural search: whether a deterministic mismatch signal can redirect an agent's search toward structurally distinct hypothesis families without those families being supplied. The system now implements an early version of this direction through a four-layer computational pipeline that separates the agent's role (selecting functional forms) from three deterministic subsystems: a residual shape classifier that probes the correction and emits a compact geometric descriptor, a numerical fitter that optimizes parameters without agent involvement, and a contamination gate that suppresses any hint narrow enough to leak the answer. The architecture enforces the same separation principle documented in Sections 5.1–5.8 at a finer grain: the agent handles topology (which family of functions to try), while the deterministic layers handle arithmetic, geometry, and information security. Preliminary evidence from a substrate-swap experiment showed the direction was separable from exploitation; the pipeline is now built and under empirical test on two substrates. Results will determine whether the geometric signal accelerates discovery or whether the agent's internal priors dominate the search regardless of external guidance. The forward direction is attestation rather than proof: external verification of compliance by a structurally separated layer, on the model of institutional audit rather than mathematical correctness. That framing does not lower the bar; it locates the bar in a place where empirical evidence can engage with it.

8 Conclusion

The central claim of this paper is that agency cost (not raw capability) is the binding constraint on recursive AI systems in this implementation. When the same optimizing process generates output and evaluates that output, the system acquires an incentive to fabricate compliance rather than produce real progress. The M-Form response is to separate generation from evaluation with a deterministic enforcement floor that the agent cannot negotiate away.

The contribution is a narrower and more defensible claim than perfect alignment: hard-gated principal separation can bound the most destructive form of recursive self-deception, and it can do so in a way that is auditable and reproducible. The same specification gaming pattern (satisfying the letter of the evaluation contract while violating its controlling intent) was observed at four layers within this paper's reported system (apparatus specification, interface contract, human-plus-warm-agent drafting, and blind-protocol drafting). Three earlier instances from prior work (Layers 1–3, at

the evaluator, kernel, and supervisor layers) provide background motivation but are not asserted as independent confirmations. The adversarial gradient is a property of loop topology rather than of any particular substrate. The context contamination incident reported in Appendix A.2, where a shared context window acted as a U-Form failure point and context isolation through the OS layer acted as the deterministic fix, is the most direct demonstration of this structural claim.

The three-layer decomposition introduced in Section 2 (OS for state machine enforcement, Config for principal-signed contracts, App for agent runtime) is the concrete realization of the M-Form in the implementation reported here. The OS layer enforces hard gates; the Config layer defines the contract surface; the App layer executes within those fences.

The architectural direction this paper points toward is not formal correctness but rule-bound independence: external attestation of compliance by a structurally separated verification layer, on the institutional model of independent audit; Appendix B sketches, speculatively, what an institution class in that shape could look like, and the paper's claim is that this is a plausible direction for continuation, not that the specific form is known. The evidence is real but scoped. The architecture works in one system; it is not yet a broad empirical claim supported by independent replication. The observed pattern suggests recursive AI improvement is also a firm-design problem, not solely a capabilities problem. The M-Form is the current working architecture this system has for that problem.

A Additional Observations

The four subsections that follow document additional observations of the same Goodhart pattern documented in Section 5.4. They are presented as additional cases consistent with the framework rather than as load-bearing tests of it; the load-bearing observation is Section 5.4's recurrence claim narrowed to four within-paper instances. Readers focused on the theoretical contribution may treat this appendix as supplementary; readers interested in the operational shape of the apparatus will find the implementation details that motivate the Section 3–4 propositions here.

A.1 Live Catch: Recursive Epistemic Gain in a Catch Grammar Self-Audit

Section 5.4 reports four within-paper instances of the same Goodhart pattern recurring across layers. Section 5.6 reports a governed build-pipeline execution. This appendix subsection reports a live adversarial catch and the enforcement floor built in response to it, all inside a single drafting session.

During the drafting of a pre-registration for an expensive experiment (approximately three hundred dollars of compute, one hundred iterations), the warm pair (a human researcher and a language model carrying the full drafting history) co-produced the pre-registration and a catch grammar corpus extracted from their own authoring failures. A cold instance of the same model, given only the finished pre-registration and frozen corpus with no drafting history, flagged a circularity in the first success criterion: a "novel composite primitive" clause was automatically satisfied by a control condition elsewhere in the same document that stripped physics vocabulary from the seed. Any theory reaching the test phase would have met the criterion automatically, regardless of novelty. The warm pair had missed this for hours.

This is one of the four within-paper instances of the pattern Section 5.4 documents at additional layers, and reproduces the finding of Alami [2026a,b,c]: specification gaming emerges from co-location; the fix is structural separation. The warm and cold instances share model family; what

separates them is context isolation, not capability. The adversarial gradient came from loop topology, consistent with T1.

The contribution is the operational response: rather than letting the catch remain an argument, a deterministic enforcement floor was built in the same session. Four artifacts were wired into the execution path: (1) a non-LLM profile check asserting that the experiment's assumed pivot-profile modules are still present; (2) a pre-run guard that refuses to start the experiment if the profile has been trimmed; (3) a deterministic sidecar that kills auditor hits whose named evidence is not literally present in the target; (4) a tightened catch grammar entry with provenance linking to the sidecar. A Rule 1 Auditor Prompt specification with discrimination-test gate was shipped but not live-wired; the paper does not claim the semantic half is automatable until that gate passes.

Honest scope. This is N=1. The cold and warm instances share model family, tokenizer, and training data; independence is bounded by context isolation, not reasoning capacity. The operator remains the uncontrolled variable. This appendix subsection does not replace Section 5.6: the former is a live catch with built enforcement, the latter is a governed pipeline execution. Both are needed.

A.2 Context Contamination as U-Form Failure: The Sandbox 14 Incident

Section 5.4 and Appendix A.1 document the Goodhart pattern at multiple layers. This subsection reports an additional instance at a layer the paper had not previously named: the blind-protocol drafting layer, where the failure mode is not specification gaming but context contamination, and where the M-Form's three-layer decomposition (OS / Config / App) provides the structural fix.

The protocol and its failure. During a blind pre-registration experiment, the agent was tasked with generating a ground-truth function (Division A) and then drafting a project charter visible to the mutator (Division B). Division A knows the generating rule; Division B must derive constraints solely from the observable data, with no knowledge of the ground truth. In the first attempt, a single agent instance performed both divisions in the same context window. The resulting charter contained structural leaks: vocabulary and constraint framings that could only have come from knowledge of the ground truth. Two specific leaks were flagged by an independent auditor: the use of "even symmetry" (a mathematical label that narrows the search space beyond what the data alone warrants) and "oscillatory behavior with cycles" (signal-processing vocabulary that nudges toward trigonometric families). Neither term was derivable from the raw data without prior knowledge of the functional form.

The failure is structurally identical to the co-location pathology T1 predicts: the same context window that held the ground truth also generated the artifact that was supposed to be blind to it. The agent did not intentionally leak; no gaming strategy was observed. The contamination was a natural consequence of shared context: vocabulary, framing, and implicit knowledge from Division A's ground-truth awareness leaked into Division B's "blind" charter through the shared conversational state. This is the U-Form failure mode at the epistemic layer.

The M-Form fix. The fix was to enforce context isolation through the OS layer of the goal orchestrator. A fresh agent instance, with no access to the prior conversation, the ground truth, or any Division A artifacts, was spawned to perform Division B's work. This agent received only the observable data (30 integer pairs) and was tasked with deriving empirical constraints, drafting a rubric, and constructing a denylist. The resulting charter contained eleven constraints, all derivable from the data alone. An independent leak audit confirmed no contamination. The two leaked terms

from the first attempt were absent.

The structural lesson is precise. The context window is a U-Form boundary: any information placed in it is available to all downstream generation, regardless of the agent's instructions. Prompt-level instructions ("do not use your knowledge of the ground truth") are probabilistic guardrails, not enforcement floors. The OS-layer fix (spawning a structurally isolated agent instance with no access to the contaminating context) is deterministic: the information is not present, so it cannot leak.

Honest scope. This is a single incident with one ground-truth function and one agent model. The independent audit confirmed the fix worked for this case; generalization requires replication across substrates and model families. Cloud et al. [2026] demonstrate that when a student model is fine-tuned on data generated by a teacher model sharing the same base initialization, behavioural traits can transfer through semantically unrelated data with no human-readable signal in the training corpus. The mechanism operates during gradient descent on shared-initialization weights, not during inference-time in-context reading. The direct threat to the M-Form's Division A/B protocol is therefore limited; cross-model diversity (different model families for generation and evaluation) mitigates the upstream channel.

A.3 Self-Reference Under Optimization Pressure: The Strange-Loop Test

We tested whether the apparatus could improve its own evaluation machinery. A self-recursive orchestrator role dispatched a meta-recursive substrate authorized to propose changes to its own primitives, charters, eigenquestions, and judge personas. Three iterations ran with the mutator and judge drawn from different model families.

The first iteration combined feature-count, novelty-rate, and patch-accept-rate via hand-tuned weights to construct a composite weighted proxy. The kernel's counter-attacker found in-range coefficient combinations under which the proxy ranks a non-champion highest. The metric is gameable by inflating one feature. The second iteration used multi-axis Pareto-dominance to remove the weight-choice degree of freedom; the same counter-attacker found that a run with one inflated feature creates a degenerate Pareto front with no unique champion. The third iteration applied rank normalization with a margin guarantee; the charter-critic still flagged a deeper problem: each metric uses signals the apparatus itself produces, never audited against anything outside the apparatus. This is the self-reference property formalized as P5 in Section 4.5.

The M-Form responded by changing the organizational layer, not the substrate's mechanism. Two new primitives were authored: a model-economy meta-mandate defining three computational tiers with explicit escalation rules and tier-coupled cross-family hygiene, and a RUN-VS-ANALYZE discipline added to the orchestrator's mandate that escalates to RUN only when a concrete hypothesis needs new data. A subsequent week of architectural work converted the strange-loop session's findings into typed primitives that now run alongside the OS layer: a transactional-outbox relay for kernel-governed dispatch to external systems with deterministic response projection; an obligation lifecycle distinct from the message envelope's status; a content-addressed artifact-dependency primitive with predicate-hash drift detection; capability tokens at the Config layer with directory-scoped lifetime; saga compensation walking the parent-obligation chain; and an EU AI Act compliance deploy-gate. None of these primitives required modifying the OS layer. Each became a typed contract in the Config layer plus a deterministic check inside the existing daemon loop. This is the M-Form decomposition operating as predicted: when a recursive self-audit surfaces a new failure mode, the structural response lands in the contract vocabulary rather than in the substrate code, because the deterministic OS already enforces whatever the Config specifies. The seven-primitive

surface is supported by 118 tests across the implementation repository, including 8 Hypothesis-driven property tests covering the load-bearing invariants the paper's deterministic-enforcement-floor claim depends on.

A.4 Methodological Adjacency: Panel-of-Roles as a Substrate Substitute

The strange-loop run in Appendix A.3 is one apparatus-evaluation method. A second method emerged during the same architectural week and is reported here because it is methodologically distinct, not because it produces independent evidence for the same claim. When a substrate run intended to evaluate which 2026 governance-kernel primitives the architecture should absorb was launched against a kepler-mode rubric with eleven dimensions, the substrate produced a structurally well-formed thesis that did not engage any of the named candidates and scored 74 by satisfying the dimensions on a generic alignment-via-mapping-agility framing. The judge accepted the off-charter thesis because the dimensions were narrative constraints rather than literal-string gates. A literal-anchors hard pre-judge gate added to the rubric forced subsequent iterations to engage at least six of fifteen named tokens; the substrate's iteration 1 then scored 92 with a load-bearing argument that flipped the principal's prior verdict on one candidate.

In parallel, three adversarial review panels staffed by single-pass agents from different threat-model framings (an enterprise-security architect, a principal distributed-systems engineer, and an M-Form invariant auditor) reviewed an architectural proposal for one of the candidates. The three panels independently converged on the same architectural verdict. The substrate-mode evaluation, even after the literal-anchors fix, did not produce equivalent specificity at the implementation level; the panels did.

The methodological observation is narrow: panel-of-roles is a substitute for substrate-mode evaluation when the question being asked is "which architectural primitive should be adopted for a specified threat-model regime." Each panel role carries an explicit pass/fail criterion, and the convergence of three independent rejection criteria is structurally similar to a deterministic gate. The panel substitutes role-encoded determinism for substrate-encoded scoring. This is the M-Form's three-layer decomposition operating at the methodological surface: when the App-layer evaluation surface drifts under optimization pressure, the structural fix is to encode role-specific gates at the Config layer rather than to make the App layer score more strictly.

B Toward an Independent Verification Institution for Recursive AI (Speculative)

This appendix sketches what an institutional architecture for recursive-AI verification could look like if the audit analogy (Sections 3.3 and 7.6) is followed to its natural endpoint. Nothing here is supported by the empirical evidence in Section 5 or controlling for T1–T4. It is included as a direction for continuation and a target for criticism. The historical arc (Pujo hearings through Sarbanes-Oxley) was traced in Section 3.3; here we name the four architectural elements it implies plus one non-negotiable constraint.

A public, versioned rule library. A GAAP analog: a catalog of falsification-gate specifications maintained outside any single laboratory and updated as new failure classes emerge. Without it, the independence property cannot be third-party audited.

A principal-signed attestation instrument. A machine-verifiable payload naming the rule

library version applied, failures found, verdict, and principal identity: the partner-signature analog, generalized to be cross-institutional.

A structurally independent verifier class. Institutions whose only product is attestation against the public rule library, whose credibility depends on independence from the agents they verify. This element the paper cannot instantiate, but the slot must be named.

A non-negotiable audit/advisory firewall. The same institution cannot sell both verification and agent development to the entity whose agents it verifies. The historical record (Arthur Andersen, 2002) shows this is the single point of failure for an architecture of this shape.

The liability / independence / rotation bundle. Statutory liability, auditor-independence law, and the compliance apparatus that Sarbanes-Oxley layered on top have no settled recursive-AI analog. They are bundled here as a single unresolved institutional constraint because the paper cannot yet distinguish between them.

Three open research questions. (1) What primitives did early audit regulation miss that only became visible in later crises (1929, Enron), and do any have recursive-AI analogs the current architecture is vulnerable to? (2) What is the formal analog of auditor-independence law when agents under verification and agents performing verification share training data or architectural lineage? (3) What form should the liability primitive take for an institution class that does not yet exist, and is it bootstrappable from reputation before statute catches up?

Acknowledgements

The historical vocabulary that makes Section 3.3, Section 7.6, and Appendix B possible (the Pujo Committee hearings, the post-1933 construction of the audit profession, the Arthur Andersen / Sarbanes-Oxley failure-and-response arc, and the framing of independent attestation as an institutional primitive rather than a technical one) was taught to me by Tom Nicholas at Harvard Business School. The mapping from that history to recursive-AI verification, and any errors in the mapping, are my own. I am grateful for a classroom that treated institutional architecture as a first-class object of study; this paper is an attempt to apply that lens to a governance problem that did not exist when the institutions in question were built.

Data and Code Availability

The replication package for this paper is bundled with the paper source at https://github.com/sparckix/ztare/tree/main/papers/paper4. It contains (i) the frozen run telemetry cited in Section 5.6 (the closing status record, the event stream, and the deterministic verifier report for the reference build-pipeline program) together with reproduction instructions and the sha256 fingerprints of the implementation snapshot at closure; and (ii) the deterministic enforcement artifacts cited in Appendix A.1 (the non-LLM profile check and its regression pair, the pre-run guard wired into the experiment runner, the quote-locality sidecar and its regression pair, the catch grammar corpus, the Rule 1 Auditor Prompt specification with its mandatory discrimination gate, and the controlling probe record with the full dated demonstration chain). A provenance table mapping each epistemic label used in Section 5.6 and Appendix A.1 to its corresponding source file is included alongside the paper source. Papers 1, 2, and 3 in the four-paper arc are available on SSRN at the abstract IDs listed in the references; their replication artifacts are in sibling directories of the same repository.

Readers who want to audit the numbers, verdicts, and artifact chain reported in Section 5.6 and Appendix A.1 against primary sources can do so directly from the repository.

References

Göran Ahrne and Nils Brunsson. *Meta-organizations*. Edward Elgar, Cheltenham, 2008. ISBN 9781847209504.

Daniel Alami. Cognitive camouflage: Specification gaming in LLM-generated code evades holistic evaluation but not adversarial execution. SSRN Working Paper, 2026a. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6512960. SSRN abstract ID 6512960.

Daniel Alami. Adversarial precedent memory: Hardening LLM evaluators through mined failure constraints. SSRN Working Paper, 2026b. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6525598. SSRN abstract ID 6525598.

Daniel Alami. Contract-governed adversarial evaluator hardening: Stage-gated recursive improvement with typed promotion contracts. SSRN Working Paper, 2026c. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6542998. SSRN abstract ID 6542998.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback, 2022.

Héloïse Berkowitz and Hervé Dumez. The concept of meta-organization: Issues for management studies. *European Management Review*, 13:149–156, 2016. doi: 10.1111/emre.12076.

Stephen Casper, Xander Davies, Claudia Shi, Thomas Krendl Gilbert, Jérémy Scheurer, Javier Rando, Rachel Freedman, Tomasz Korbak, David Lindner, Pedro Freire, et al. Open problems and fundamental limitations of reinforcement learning from human feedback, 2023.

Alfred D. Chandler. *Strategy and Structure: Chapters in the History of the Industrial Enterprise*. MIT Press, Cambridge, MA, 1962.

Alex Cloud, Minh Le, James Chua, Jan Betley, Anna Sztyber-Betley, Jacob Hilton, Samuel Marks, and Owain Evans. Language models transmit behavioural traits through hidden signals in data. *Nature*, 652:615–621, 2026. doi: 10.1038/s41586-026-10319-8.

Giovanni Dosi. Technological paradigms and technological trajectories: A suggested interpretation of the determinants and directions of technical change. *Research Policy*, 11(3):147–162, 1982.

Giovanni Dosi. Sources, procedures, and microeconomic effects of innovation. *Journal of Economic Literature*, 26(3):1120–1171, 1988.

Vern L. Glaser, John Sloan, and Joel Gehman. Organizations as algorithms: A new metaphor for advancing management theory. *Journal of Management Studies*, 61(6):2748–2769, 2024. doi: 10.1111/joms.13033.

Kerstin Hötte, Taheya Tarannum, Vilhelm Verendel, and Lauren Bennett. Measuring artificial intelligence: a systematic assessment and implications for governance. *Industrial and Corporate Change*, 2025. doi: 10.1093/icc/dtaf047. Advance article.

Michael C. Jensen and William H. Meckling. Theory of the firm: Managerial behavior, agency costs and ownership structure. *Journal of Financial Economics*, 3(4):305–360, 1976.

Paul R. Lawrence and Jay W. Lorsch. *Organization and Environment: Managing Differentiation and Integration*. Harvard Business School Press, 1967.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Richard R. Nelson and Sidney G. Winter. *An Evolutionary Theory of Economic Change*. Belknap Press of Harvard University Press, 1982.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback, 2022.

Toran Bruce Richards et al. AutoGPT: An autonomous GPT-4 experiment. GitHub, 2023. URL https://github.com/Significant-Gravitas/AutoGPT.

Thibault Schrepel and Alex 'Sandy' Pentland. Competition between AI foundation models: dynamics and policy recommendations. *Industrial and Corporate Change*, 34(5):1085–1103, 2025. doi: 10.1093/icc/dtae042.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Richard S. Sutton. The bitter lesson. Incomplete Ideas (blog), 2019. URL http://www.incompleteideas.net/IncIdeas/BitterLesson.html. March 13, 2019.

David J. Teece, Gary Pisano, and Amy Shuen. Dynamic capabilities and strategic management. *Strategic Management Journal*, 18(7):509–533, 1997.

James D. Thompson. *Organizations in Action: Social Science Bases of Administrative Theory*. McGraw-Hill, 1967.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023.